FULL STACK DEVELOPMENT WITH MERN

HotelPro

Enterprise Hotel Booking & Management Solution

PROJECT DOCUMENTATION

Submitted by	Aditi Yadav
Program	B.Tech Computer Science & Engineering, 5th Semester (Session 2026-27)
Institution	Faculty of Engineering, Medicaps University
Technology stack	MongoDB, Express.js, React.js, Node.js (MERN)
Source code	https://github.com/AditiYadav-03/HotelPro
Live demo	https://hotel-pro-taupe.vercel.app

Table of Contents

1. Introduction 4

1.1 Project Title 4

1.2 Team Members 4

1.3 Project Links 4

2. Project Overview 4

2.1 Purpose 4

2.2 User Roles 5

2.3 Key Features 5

3. Architecture 6

3.1 Frontend (React) 6

3.2 Backend (Node.js and Express.js) 6

3.3 Database (MongoDB) 7

4. Setup Instructions 7

4.1 Prerequisites 7

4.2 Installation 8

5. Folder Structure 9

5.1 Client (React Frontend) 9

5.2 Server (Node.js Backend) 10

6. Running the Application 10

6.1 Start the Backend Server 10

6.2 Start the Frontend Client 10

7. API Documentation 11

7.1 Authentication (/api/auth) 11

7.2 Rooms and Inventory (/api/rooms) 11

7.3 Bookings and Reservations (/api/bookings) 11

7.4 Payments and Invoices (/api/payments) 12

7.5 Hotel Operations and Housekeeping (/api/operations) 12

7.6 Guest Services (/api/services) 12

7.7 Ratings and Reviews (/api/reviews) 12

7.8 Admin Analytics and Logs (/api/admin) 13

7.9 Example Requests and Responses 13

8. Authentication and Authorization 14

8.1 Authentication Flow 14

8.2 Authorization (RBAC) . . . 14
8.3 Demo Accounts . . . 15

9. User Interface . . . 15
9.1 Public Pages . . . 15
9.2 Guest Dashboard . . . 16
9.3 Staff Dashboard . . . 17
9.4 Admin Dashboard . . . 18

10. Testing . . . 19
10.1 Testing Strategy . . . 19
10.2 Tools . . . 19
10.3 Sample Test Cases . . . 20

11. Screenshots and Demo . . . 20
11.1 Live Demo . . . 20
11.2 Suggested Demonstration Flow . . . 20
11.3 Screenshots . . . 21

12. Known Issues . . . 22

13. Future Enhancements . . . 23

1. Introduction

1.1 Project Title

HotelPro: Enterprise Hotel Booking & Management Solution

HotelPro is a production-style hotel booking and management web platform built on the MERN stack. It brings the guest-facing booking journey and the hotel's internal operations (front desk, housekeeping and management reporting) into a single application.

1.2 Team Members

Name	Role	Responsibilities
Aditi Yadav	Full Stack Developer	Requirement analysis, React frontend, Node.js/Express backend, MongoDB schema design, authentication, testing and deployment

1.3 Project Links

- **Source repository:** https://github.com/AditiYadav-03/HotelPro
- **Deployed application:** https://hotel-pro-taupe.vercel.app

2. Project Overview

2.1 Purpose

The purpose of HotelPro is to digitise end-to-end hospitality operations. Traditional hotel workflows often depend on separate tools for reservations, billing, front-desk work and housekeeping, which leads to double bookings, slow check-ins and limited visibility for management. HotelPro addresses this with one role-aware platform.

The main goals of the project are:

- Allow guests to search, filter, book and pay for rooms online without conflicts or double bookings.
- Give front-desk staff a digital desk for check-in, keycard issuance, check-out and housekeeping coordination.
- Provide administrators with revenue, occupancy and security-audit insights for decision making.
- Enforce secure, role-based access so each user sees only what their role permits.

2.2 User Roles

Role	Primary Purpose	Key Capabilities
Guest	Book and manage stays	Room search, booking and payment, invoice printing, room-service requests, reviews
Hotel Staff	Run daily operations	Check-in desk, keycard issuance, check-out inspection, housekeeping board, service request dispatch
Admin	Govern and analyse the business	Financial analytics, room inventory management, user role management, audit logs

2.3 Key Features

The functionality is organised into six epics covering 19 tasks.

Epic	Scope and Implemented Features
1. Authentication & Security	User registration, JWT login, bcrypt password hashing, session recovery, and role-based access control (RBAC) middleware for Guest, Staff and Admin.
2. Hotel & Inventory Management	Room category schemas (Deluxe, Suite and others), room inventory CRUD, multi-image gallery support, floor allocation and amenity tagging.
3. Guest Reservation Workflow	Room filtering by price range, capacity and category; date-range collision detection engine; booking confirmation; cancellation with refund calculation.
4. Payment & Invoicing	Simulated payment gateway flow, transaction ID tracking, and downloadable or printable tax invoices with GST breakdown.
5. Hotel Operations & Staff Desk	Digital check-in with keycard and ID proof validation, check-out room condition inspection, live housekeeping status board and guest service request dispatch.
6. Analytics, Reviews & Admin Control	Business KPIs (total revenue, occupancy rate), Recharts financial trend charts, star reviews with staff replies, notifications and security audit logs.

Highlights

- Real-time room collision detection to prevent double-booking.
- Dynamic price calculation including GST breakdown and seasonal discounts.
- Automated invoice generation for each reservation.
- Interactive executive dashboard built with Recharts.

3. Architecture

HotelPro follows a classic three-tier MERN architecture. The React single-page application communicates with a stateless Express REST API over HTTP, and the API persists data in MongoDB through Mongoose.

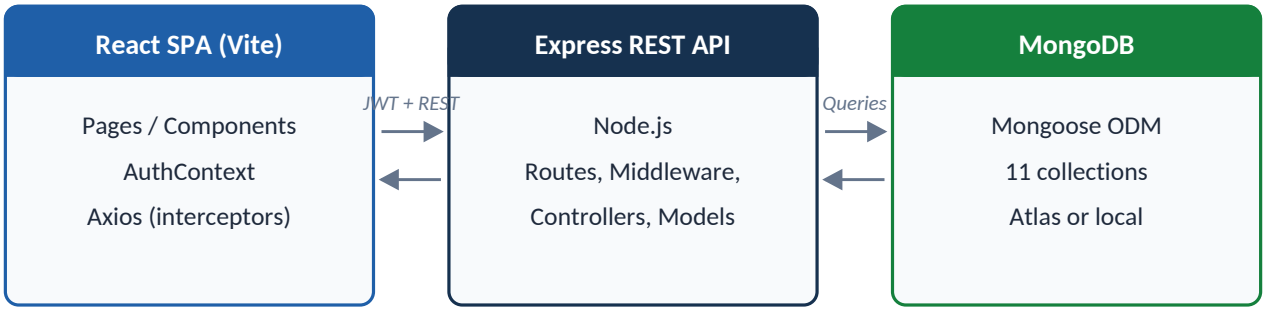


Figure 3.1: Three-tier MERN architecture of HotelPro

3.1 Frontend (React)

The client is a React application bootstrapped with Vite and structured around reusable components and page-level views.

- **Routing:** React Router (configured in `App.jsx`) maps public pages (Home, Room List, Room Details, Login, Register) and role-specific dashboards.
- **Protected routes:** a `ProtectedRoute` component restricts dashboards to authenticated users with the correct role.
- **Global state:** `AuthContext` stores the signed-in user and JWT, and restores the session when the application reloads.
- **API layer:** a shared Axios instance with interceptors in `services/` attaches the token to requests and centralises error handling.
- **Reusable UI:** Navbar, Footer and modal components for invoices, reviews and service requests.
- **Data visualisation:** Recharts renders revenue trends and occupancy distribution on the admin dashboard.

3.2 Backend (Node.js and Express.js)

The server exposes a modular REST API. Each request passes through routing, middleware, a controller that holds the business logic, and a Mongoose model.

- **Routes:** separate routers for `/api/auth`, `/api/rooms`, `/api/bookings`, `/api/payments`, `/api/operations`, `/api/services`, `/api/reviews` and `/api/admin`.
- **Middleware:** JWT verification, role-based authorisation (RBAC), centralised error handling and Multer for image uploads.
- **Controllers:** implement the domain rules, such as date-range collision detection, price and GST computation, refund calculation, check-in and check-out workflows.
- **Utilities:** a database seeder that loads rooms, categories, demo users, bookings, payments and reviews.
- **Entry point:** `server.js` configures Express, connects to MongoDB and mounts the routers.

3.3 Database (MongoDB)

MongoDB stores all application data. Mongoose defines 11 schemas, including User, Room, Booking and Payment. Collections are connected through document references, for example a booking references its guest and room, and a payment references its booking. The principal entities are summarised below.

Entity	Purpose	Typical Interactions
User	Stores account details, hashed password and role (Guest, Staff, Admin).	Registration, login, profile update, admin role management
Room / Room Category	Room inventory with category, price, capacity, floor, amenities, images and status.	Search and filter, admin CRUD, housekeeping status updates
Booking	A reservation for a room over a date range, with pricing and status.	Created by guests with collision checks; cancelled with refund; checked in and out by staff
Payment	Transaction record with transaction ID and amount breakdown.	Created at payment; used to produce the invoice
Review	Star rating and comment for a room, with optional staff reply.	Guest submission, staff response, public listing per room
Service Request	Room dining, housekeeping or maintenance requests from guests.	Created by guests; dispatched and updated by staff
Notification and Audit Log	System notifications and a security audit trail of key actions.	Surfaced to users and reviewed by administrators

> **Note:** the entity table lists the main concepts the application manages. Field-level details are defined in the Mongoose schema files under `backend/models/`.

4. Setup Instructions

4.1 Prerequisites

Software	Requirement	Notes
Node.js	v16 or above	Verify with `node -v`
npm	Bundled with Node.js	Used for dependency installation and scripts
MongoDB	Atlas (cloud) or local Community Server	Atlas free M0 cluster is the recommended option
Git	Any recent version	For cloning the repository
Web browser	Any modern browser	Chrome, Edge or Firefox

4.2 Installation

Step 1: Clone the repository

```
git clone https://github.com/AditiYadav-03/HotelPro.git
cd HotelPro
```

Step 2: Install dependencies

```
cd backend
npm install
cd ../frontend
npm install
```

Step 3: Configure the database

Option A: MongoDB Atlas (recommended)

1. Create a free account on MongoDB Atlas and build a free **M0 cluster**.
2. Under **Database Access**, create a database user with a strong password.
3. Under **Network Access**, add an IP address. For development, **Allow Access from Anywhere (0.0.0.0/0)** can be used.
4. Choose **Connect, then Drivers** and copy the connection string.
5. Paste it into `backend/.env` as the `MONGO_URI` value.

Option B: Local MongoDB

If MongoDB Community Server is installed locally, use the default connection string:

```
MONGO_URI=mongodb://127.0.0.1:27017/hotelpro
```

Step 4: Set environment variables

Create a file named `.env` inside the `backend` folder.

Variable	Description	Example
`MONGO_URI`	MongoDB connection string	`mongodb+srv://<user>:<password>@cluster0.mongodb.net/hotelpro`
`PORT`	Port on which the API listens	`5000`
`JWT_SECRET`	Secret key used to sign authentication tokens	A long random string

> **Important:** never commit the `.env` file to version control. It is excluded through `.gitignore`.

Step 5: Seed sample data

Populate rooms, categories, demo users, bookings, payments and reviews with a single command from the project root (or from the `backend` folder):

```
npm run seed
```

5. Folder Structure

```
HotelPro/
├── backend/
│   ├── config/          # MongoDB connection configuration
│   ├── controllers/     # Route handlers and business logic
│   ├── middleware/      # JWT auth, RBAC, error handler, Multer
│   ├── models/          # 11 Mongoose schemas
│   ├── routes/          # REST API endpoints
│   ├── utils/           # Database seeder script and mock data
│   ├── server.js        # Entry point of the backend API
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── components/  # Navbar, Footer, ProtectedRoute, modals
│   │   ├── context/     # AuthContext (JWT and session state)
│   │   ├── pages/       # Home, rooms, auth pages, dashboards
│   │   ├── services/    # Axios instance with interceptors
│   │   ├── App.jsx      # React Router configuration
│   │   └── main.jsx     # Application bootstrap
│   ├── package.json
│   └── vite.config.js
├── README.md
├── run-hotelpro.bat     # Windows helper script
└── package.json         # Root scripts (development and seeding)
```

5.1 Client (React Frontend)

Folder / File	Description
`src/components/`	Reusable interface elements: Navbar, Footer, ProtectedRoute and the Invoice, Review and Service Request modals.
`src/context/`	`AuthContext` manages the JWT, logged-in user and global session state.
`src/pages/`	Screen-level views: Home, Room List, Room Details, Login, Register and the role dashboards.
`src/services/`	Pre-configured Axios instance with request and response interceptors.
`App.jsx`	Declares all routes and applies route protection.
`main.jsx`	Mounts the React application in the DOM.
`vite.config.js`	Vite build and development server configuration.

5.2 Server (Node.js Backend)

Folder / File	Description
`config/`	Database connection setup using Mongoose.
`controllers/`	Business logic for authentication, rooms, bookings, payments, operations, services, reviews and admin analytics.
`middleware/`	JWT verification, role-based authorisation, error handling and Multer file-upload handling.
`models/`	The 11 Mongoose schemas that define the data model.
`routes/`	Express routers that map endpoints to controllers.
`utils/`	Seeder script and mock data for quick environment setup.
`server.js`	Creates the Express app, connects to the database and starts the server.

6. Running the Application

Run the backend and the frontend in two separate terminal windows. Make sure the database is reachable and the sample data has been seeded (see Section 4).

6.1 Start the Backend Server

```
cd backend
npm run dev
```

The API runs at **http://localhost:5000**.

6.2 Start the Frontend Client

```
cd frontend
npm run dev
```

The client runs at **http://localhost:5173**. Open this address in a browser to use the application.

> **Note:** this project uses Vite, so the development server is started with `npm run dev` instead of `npm start`. On Windows, the `run-hotelpro.bat` script at the repository root is provided as a convenience launcher.

7. API Documentation

All endpoints are prefixed with the server base URL (`http://localhost:5000`). Requests and responses use JSON. Protected endpoints require the header `Authorization: Bearer <token>`.

7.1 Authentication (/api/auth)

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register a new Guest or Staff account
POST	/api/auth/login	Public	Authenticate and receive a JWT bearer token
GET	/api/auth/profile	Authenticated	Get the logged-in user's profile
PUT	/api/auth/profile	Authenticated	Update profile details
GET	/api/auth/users	Admin	List all registered users
PUT	/api/auth/users/:id/role	Admin	Update a user's role and permissions

7.2 Rooms and Inventory (/api/rooms)

Method	Endpoint	Access	Description
GET	/api/rooms	Public	Search and filter rooms by category, price, capacity and status
GET	/api/rooms/:id	Public	Get details of a single room
POST	/api/rooms/:id/availability	Public	Validate availability for a date range
POST	/api/rooms	Admin, Staff	Create a new room
PUT	/api/rooms/:id	Admin, Staff	Update room pricing and information
DELETE	/api/rooms/:id	Admin	Delete a room

7.3 Bookings and Reservations (/api/bookings)

Method	Endpoint	Access	Description
POST	/api/bookings	Guest	Create a booking with collision checks
GET	/api/bookings/my	Guest	Booking history of the logged-in user
GET	/api/bookings/:id	Authenticated	Details of a single booking
PUT	/api/bookings/:id/cancel	Guest, Admin	Cancel a reservation and process the refund
GET	/api/bookings	Staff, Admin	List all bookings across the hotel

7.4 Payments and Invoices (/api/payments)

Method	Endpoint	Access	Description
POST	/api/payments/process	Authenticated	Process an online payment transaction
GET	/api/payments/invoice/:bookingId	Authenticated	Get the printable invoice and cost breakdown

7.5 Hotel Operations and Housekeeping (/api/operations)

Method	Endpoint	Access	Description
POST	/api/operations/check-in	Staff	Check in a guest, issue a keycard and record ID proof
POST	/api/operations/check-out	Staff	Check out a guest, inspect the room and trigger cleaning
PUT	/api/operations/room/:roomId/status	Staff	Update the housekeeping status of a room
GET	/api/operations/overview	Staff	Arrivals, departures and occupancy metrics

7.6 Guest Services (/api/services)

Method	Endpoint	Access	Description
POST	/api/services	Guest	Request room dining, housekeeping or maintenance
GET	/api/services/my	Guest	View the guest's service history
GET	/api/services	Staff, Admin	Service dispatch queue
PUT	/api/services/:id	Staff, Admin	Update the status of a request

7.7 Ratings and Reviews (/api/reviews)

Method	Endpoint	Access	Description
POST	/api/reviews	Guest	Submit a rating and comment for a room
GET	/api/reviews/room/:roomId	Public	Get all reviews of a room
POST	/api/reviews/:id/respond	Staff, Admin	Reply to a customer review

7.8 Admin Analytics and Logs (/api/admin)

Method	Endpoint	Access	Description
GET	/api/admin/analytics	Admin	Revenue, occupancy and trend KPIs
GET	/api/admin/logs	Admin	Security audit trail

7.9 Example Requests and Responses

The examples below are illustrative. Exact field names and values depend on the controller implementation.

Login

```
POST /api/auth/login
Content-Type: application/json

{ "email": "guest@hotelpro.com", "password": "guest123" }
```

Example success response (200 OK):

```
{
  "token": "<JWT>",
  "user": {
    "_id": "…",
    "name": "Guest User",
    "email": "guest@hotelpro.com",
    "role": "guest"
  }
}
```

Create a booking

```
POST /api/bookings
Authorization: Bearer <token>

{ "room": "<roomId>", "checkIn": "2026-11-10", "checkOut": "2026-11-13", "guests": 2 }
```

Example success response (201 Created):

```
{
  "_id": "…",
  "room": "<roomId>",
  "checkIn": "2026-11-10",
  "checkOut": "2026-11-13",
  "status": "confirmed",
  "totalAmount": 0
}
```

Common error responses

Status	Meaning	Typical Cause
400	Bad Request	Missing or invalid input fields
401	Unauthorized	Missing, invalid or expired token
403	Forbidden	The user's role is not permitted to use the endpoint
404	Not Found	Requested room, booking or user does not exist
409	Conflict	Requested dates overlap with an existing booking
500	Server Error	Unexpected failure on the server

8. Authentication and Authorization

HotelPro uses **stateless, token-based authentication** with JSON Web Tokens (JWT) and enforces permissions through **role-based access control (RBAC)**.

8.1 Authentication Flow

1. A user registers; the password is hashed with **bcrypt** before being stored. Plain-text passwords are never saved.
2. On login, the server verifies the credentials against the stored hash and issues a signed **JWT** containing the user's identity and role.
3. The client keeps the token in `AuthContext` and the Axios interceptor attaches it to each request as a `Bearer` token.
4. Protected routes on the server pass through authentication middleware that verifies the token signature and loads the user.
5. When the application reloads, `AuthContext` recovers the session so the user does not need to sign in again while the token is valid.

8.2 Authorization (RBAC)

After authentication, authorisation middleware checks whether the user's role may access the requested endpoint. Unauthorised requests are rejected with `401` or `403` responses. The frontend additionally uses `ProtectedRoute` to hide dashboards from users without the correct role.

Capability	Guest	Staff	Admin
Search rooms, view reviews	**Yes**	**Yes**	**Yes**
Book, pay and cancel own reservations	**Yes**	No	**Cancel any**
Check-in, check-out and keycard issuance	No	**Yes**	No
Update housekeeping status	No	**Yes**	No
Create or edit rooms	No	**Yes**	**Yes**

Capability	Guest	Staff	Admin
Delete rooms	No	No	**Yes**
Reply to reviews	No	**Yes**	**Yes**
Manage user roles	No	No	**Yes**
View analytics and audit logs	No	No	**Yes**

8.3 Demo Accounts

The seeded database includes one account per role for evaluation. The login page also offers one-click demo buttons that pre-fill these credentials.

Role	Email	Password	Dashboard Access
Executive Admin	`admin@hotelpro.com`	`admin123`	Analytics, room inventory, user roles, audit logs
Front Desk Staff	`staff@hotelpro.com`	`staff123`	Check-in, keycards, check-out, housekeeping, service requests
Verified Guest	`guest@hotelpro.com`	`guest123`	Room search, booking and payment, invoices, service requests, reviews

> **Security note:** these credentials exist only for demonstration and must be changed or removed in any real deployment.

9. User Interface

The interface is a responsive single-page application. Navigation adapts to the signed-in role, and each role lands on a dedicated dashboard.

9.1 Public Pages

- **Home:** introduces the hotel and links to room search.
- **Room List:** filterable catalogue (category, price range, capacity) of available rooms.
- **Room Details:** image gallery, amenities, pricing, availability check and reviews.
- **Login and Register:** authentication forms; the login page includes one-click demo buttons.

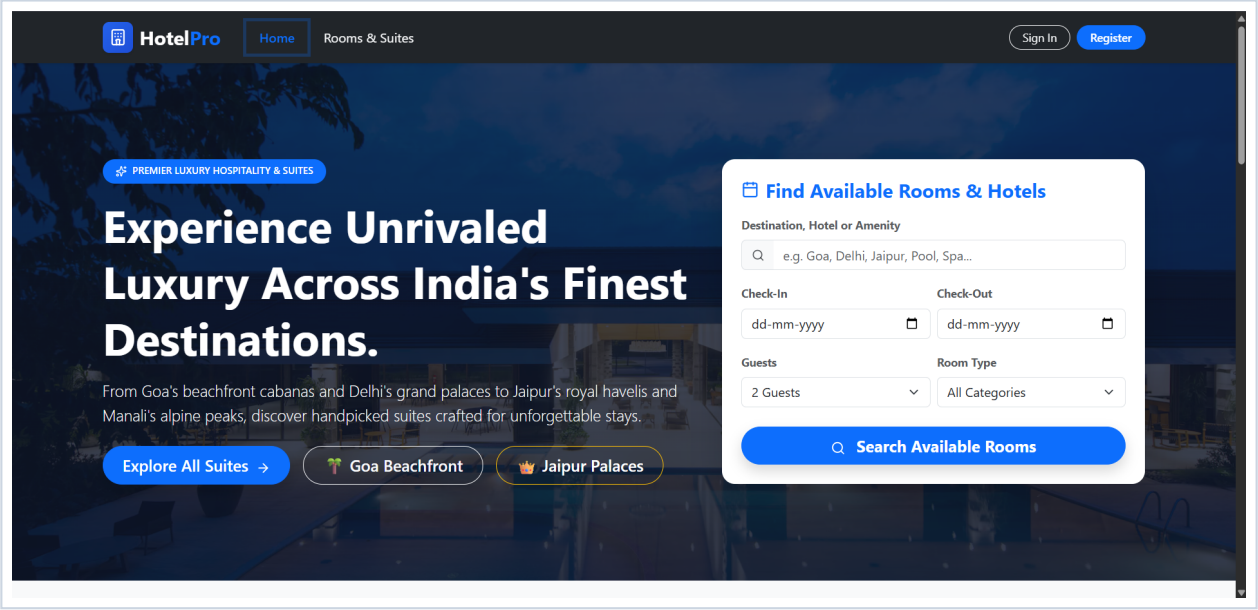


Figure 9.1: Home page

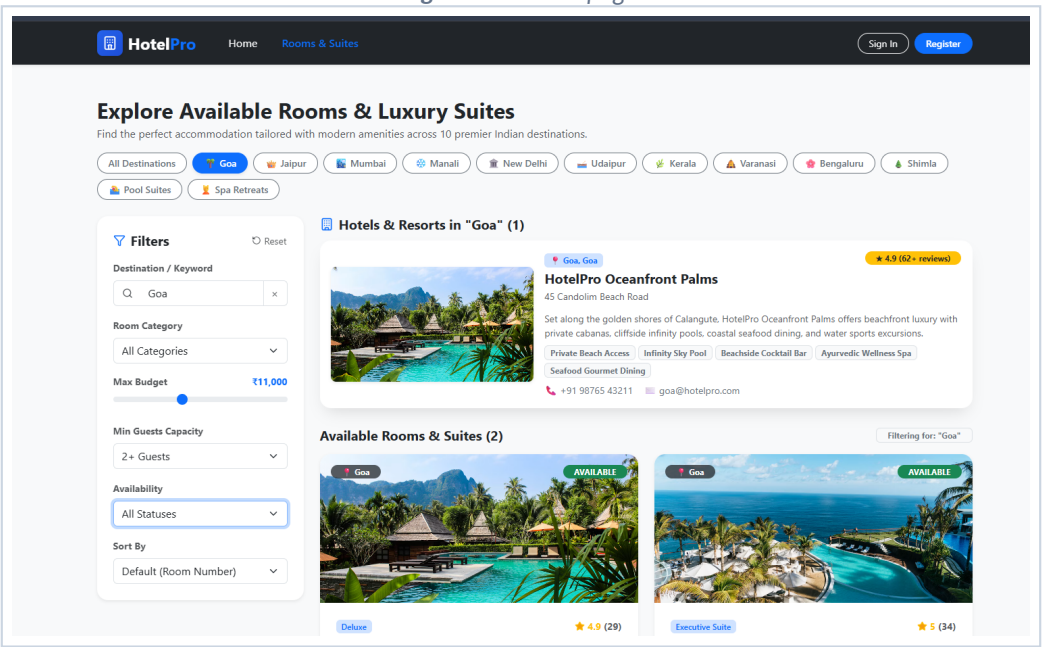


Figure 9.2: Room search and filters

9.2 Guest Dashboard

- Booking flow with price breakdown including GST and discounts.
- Payment step and printable invoice modal.
- Booking history with cancellation option.
- Room service requests and review submission modals.

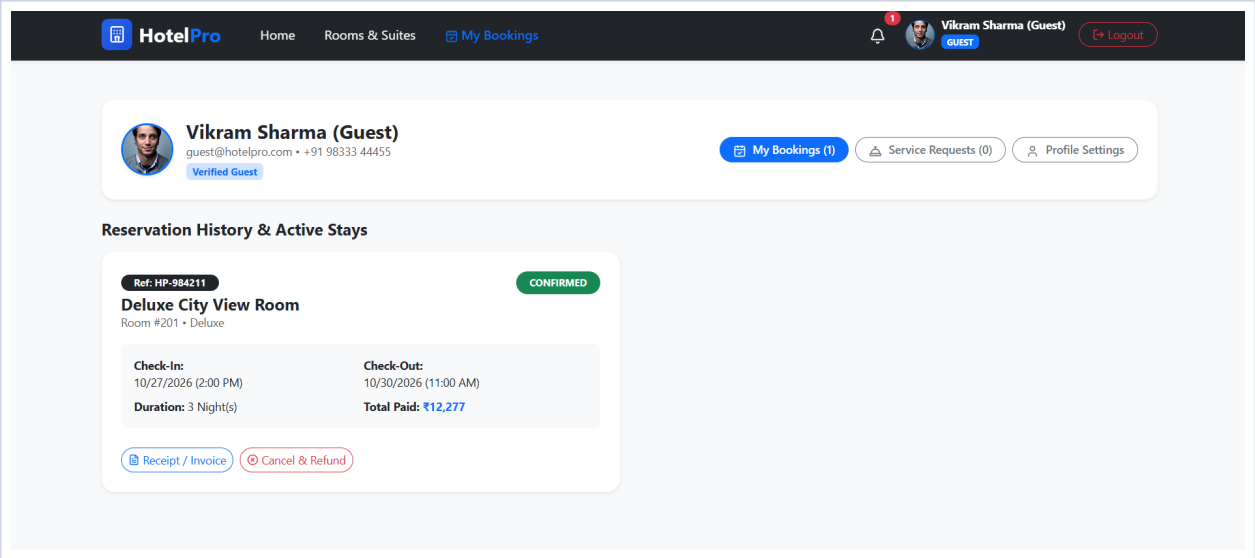


Figure 9.3: Guest dashboard

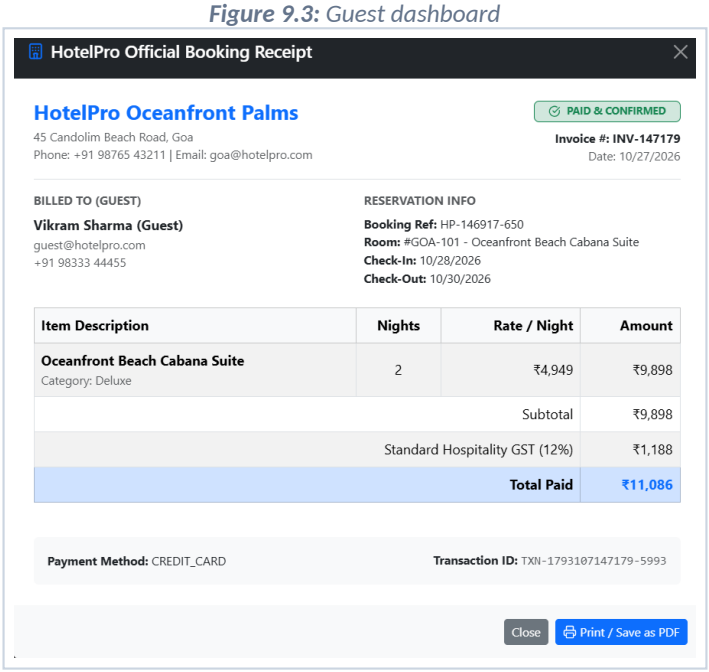


Figure 9.4: Printable invoice

9.3 Staff Dashboard

- Today's arrivals, departures and occupancy overview.
- Digital check-in with ID proof validation and RFID keycard issuance.
- Check-out with room condition inspection.
- Live housekeeping status board and guest service dispatch queue.

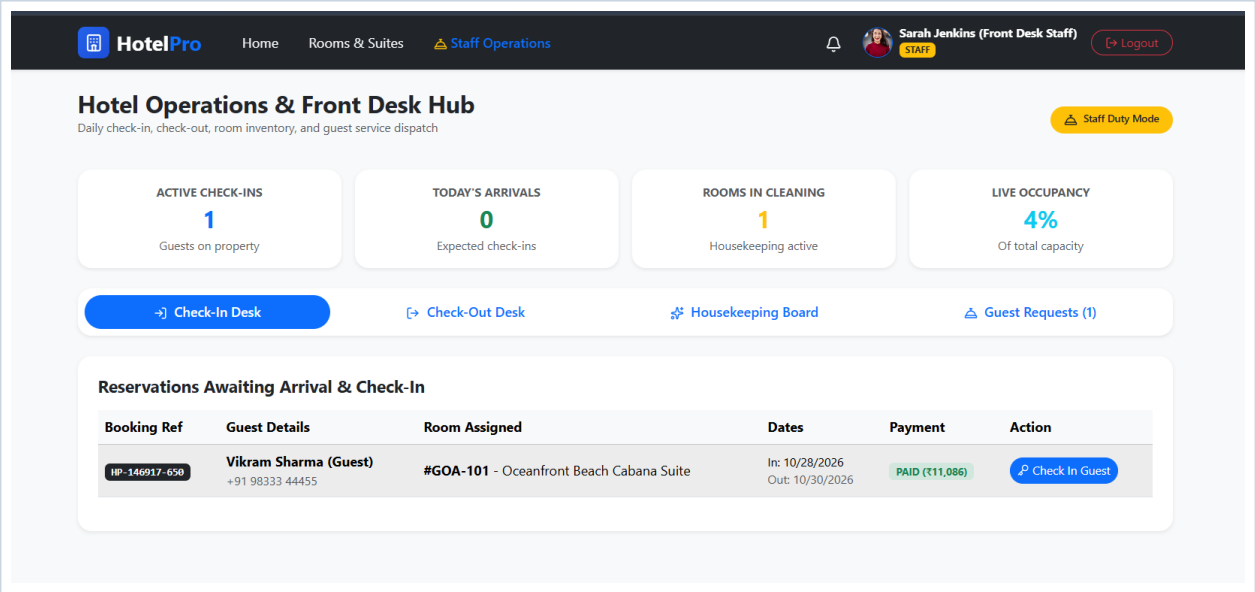


Figure 9.5: Staff operations dashboard

9.4 Admin Dashboard

- KPI cards for total revenue and occupancy rate.
- Recharts visualisations of monthly revenue and occupancy distribution.
- Room inventory management, user role management and the security audit log.

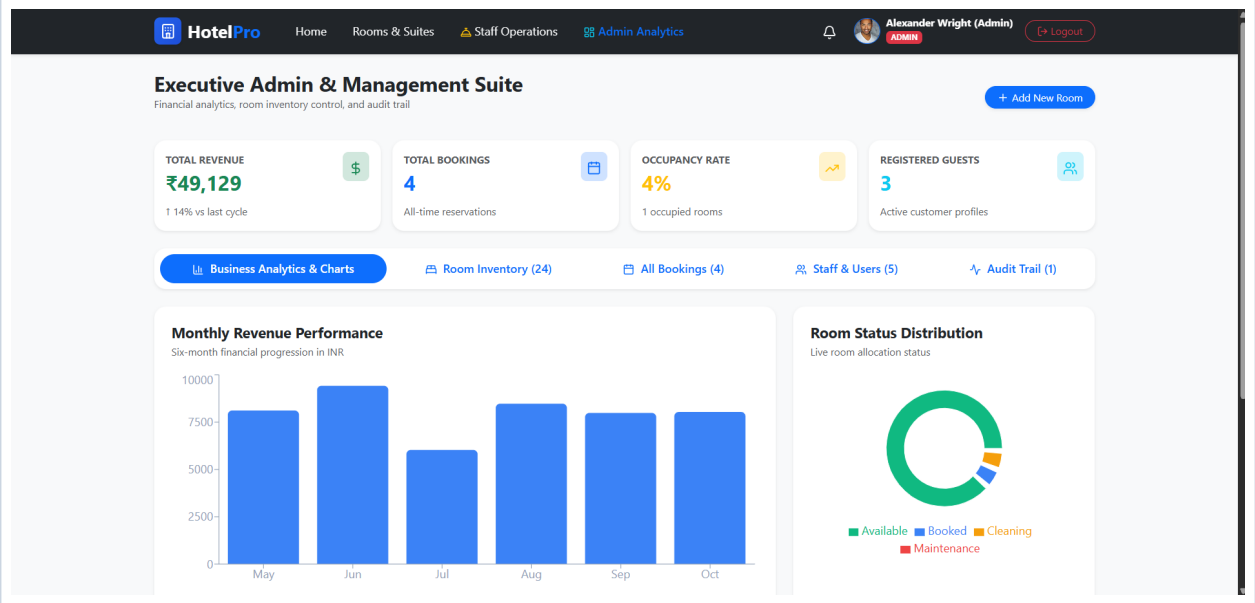


Figure 9.6: Admin analytics dashboard

10. Testing

10.1 Testing Strategy

Testing focuses on verifying that each epic behaves correctly for every user role. The seeded demo data provides a repeatable starting state, so test cases can be re-run after resetting the database with `npm run seed`.

- **Functional testing:** manual walkthrough of every user story in the browser for each role.
- **API testing:** endpoints exercised with an API client such as Postman or Thunder Client, with and without a valid token.
- **Role-based access testing:** confirm that restricted endpoints and dashboards reject users of the wrong role.
- **Boundary and negative testing:** invalid dates, overlapping bookings, unavailable rooms and wrong credentials.
- **Usability testing:** responsive layout checks on desktop and mobile screen sizes.

10.2 Tools

Tool	Purpose
Web browser developer tools	Inspect network calls, console output and responsive layouts
Postman or Thunder Client	Send REST requests and inspect status codes and payloads
MongoDB Atlas / Compass	Verify stored documents after each operation
Seed script (`npm run seed`)	Reset the database to a known state before testing

10.3 Sample Test Cases

ID	Scenario	Expected Result
TC-01	Login with correct guest credentials	Token issued; guest dashboard opens
TC-02	Login with a wrong password	Request rejected with an error message
TC-03	Guest opens the admin analytics URL	Access denied (403) or redirected
TC-04	Book a room for free dates	Booking confirmed with price and GST breakdown
TC-05	Book the same room for overlapping dates	Rejected by the collision detection engine
TC-06	Cancel a confirmed booking	Status updated to cancelled; refund calculated
TC-07	Process a payment for a booking	Transaction ID recorded; invoice available
TC-08	Staff checks in a guest with keycard	Booking marked checked-in; keycard recorded
TC-09	Staff checks out and marks room for cleaning	Room moves to the housekeeping queue
TC-10	Admin changes a user’s role	New permissions apply on the next login
TC-11	Guest submits a review; staff replies	Review and reply visible on the room page

11. Screenshots and Demo

11.1 Live Demo

The deployed application can be accessed at https://hotel-pro-taupe.vercel.app. Use the demo accounts listed in Section 8.3 to explore each role.

11.2 Suggested Demonstration Flow

1. Open the application and sign in as the **Guest** using the one-click demo button.
2. Search for a room, apply filters, view the details and complete a booking and payment.
3. Open and print the generated invoice.
4. Sign in as **Staff** to check the guest in, issue a keycard and update housekeeping status.
5. Sign in as **Admin** to review revenue and occupancy analytics and the audit log.

11.3 Screenshots

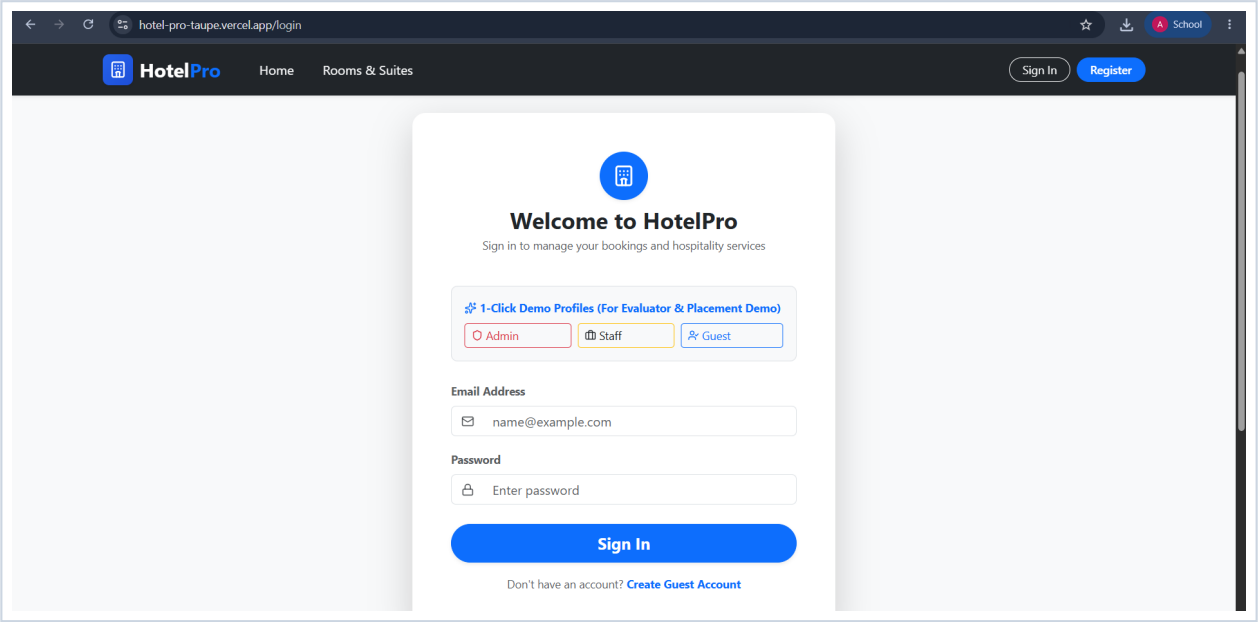


Figure 11.1: Login page

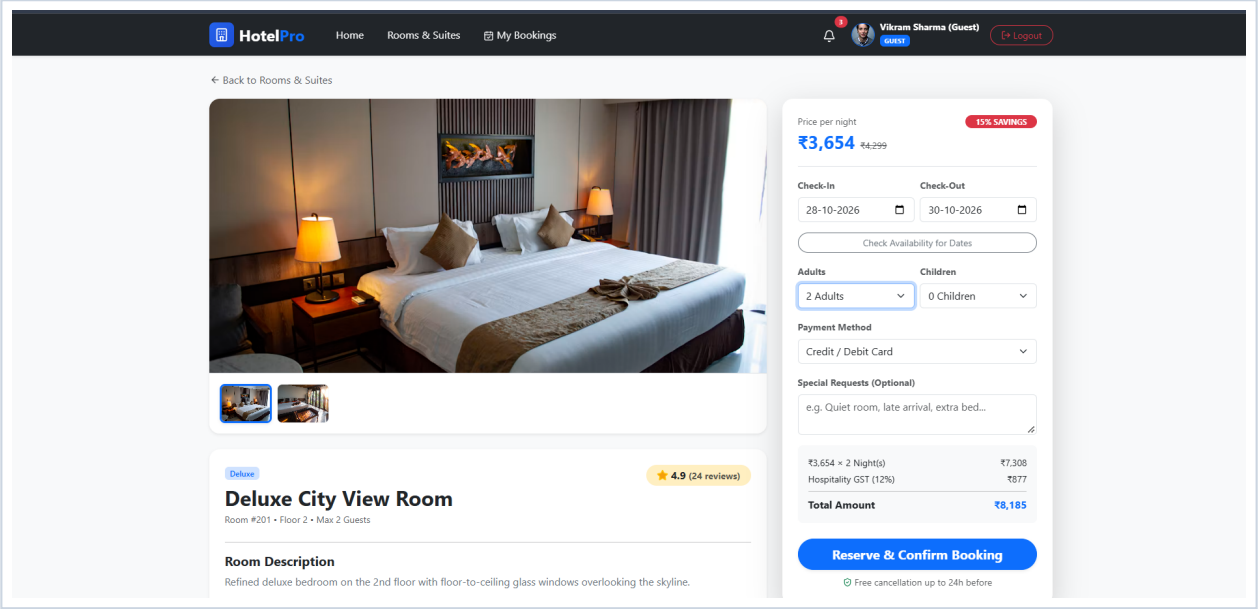


Figure 11.2: Booking and payment

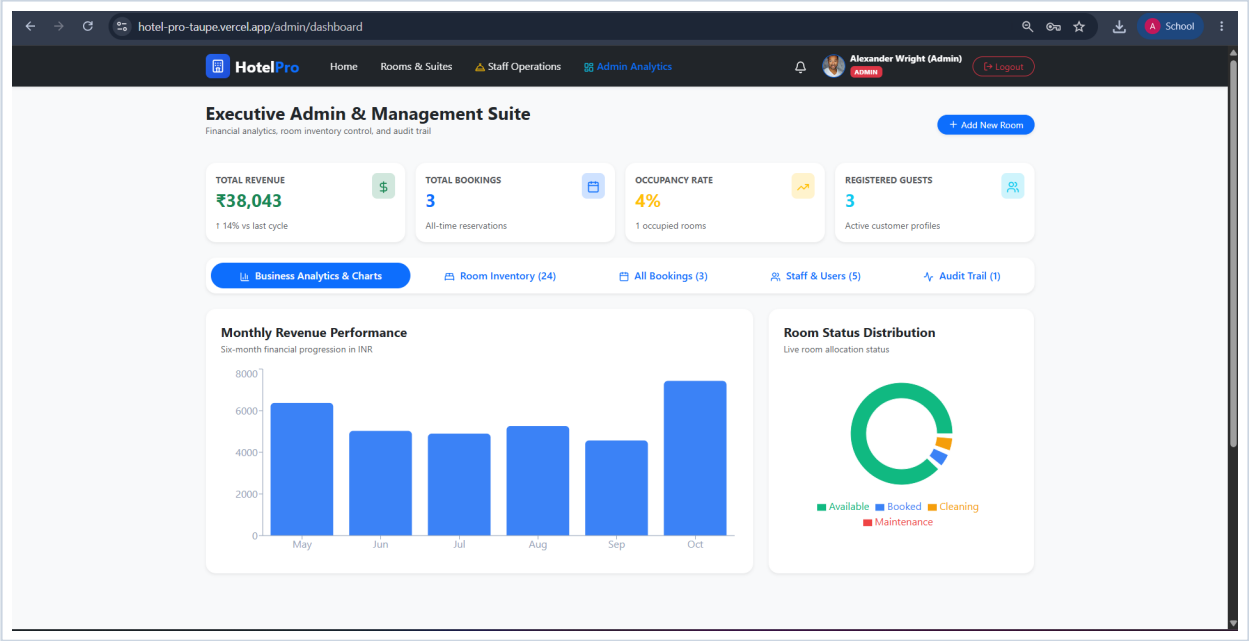


Figure 11.3: Admin analytics

12. Known Issues

The following limitations are known and should be considered by users and developers:

#	Issue	Impact / Workaround
1	The payment gateway is simulated; no real money is processed.	Suitable for demonstration only. A real provider must be integrated before production use.
2	Demo credentials are public and seeded data is mock data.	Remove or change the demo accounts and run without the seeder in production.
3	Atlas network access of `0.0.0.0/0` is convenient for development but permissive.	Restrict to specific IP addresses in production.
4	No automated test suite is bundled with the project.	Regression testing is manual (see Section 10).
5	The `run-hotelpro.bat` helper script works only on Windows.	On macOS or Linux, start the servers manually (see Section 6).
6	Commands differ from the generic MERN template (`npm run dev` instead of `npm start`).	Follow the commands in Section 6.

13. Future Enhancements

Potential improvements to extend HotelPro include:

- **Real payment integration:** connect a gateway such as Razorpay or Stripe with webhooks and refunds.
- **Notifications:** email and SMS confirmations, reminders and check-in details.
- **Stronger session security:** refresh tokens, HttpOnly cookies, rate limiting and account lockout.
- **Automated testing and CI/CD:** unit and integration tests, plus a pipeline that tests and deploys on each commit.
- **Real-time updates:** WebSockets for live room status and housekeeping changes.
- **Dynamic pricing:** demand-based and seasonal rate rules managed from the admin panel.
- **Multi-property support:** manage several hotels or branches from one account.
- **Mobile application:** a React Native or Progressive Web App version for guests and staff.
- **Containerisation:** Docker and Docker Compose for one-command local setup.
- **Localisation and accessibility:** multi-language support and WCAG-compliant interface improvements.